

Unit – III

SQL Relationships, Joins, and Subqueries

Part 1: SQL Relationships

What You Will Learn

- The meaning and usage of relationships in SQL.
- Types of relationships, including One-to-One, One-to-Many, and Many-to-Many.

What is a Relationship in SQL?

In database operations, real-world applications require combining multiple tables to complete tasks. A Relational Database Management System (RDBMS), such as MySQL, relies on relationship design to define how data across different tables connect.

In SQL, establishing a relationship does not require a new statement; instead, it relies on setting correct field correspondences (for example, the `id` column of a `users` table corresponding to the `user_id` column of an `orders` table).

1. One to One

Concept: Each record in Table A corresponds to exactly one record in Table B, and vice versa.

Example: Each transportation is responsible for only one order, and each order is assigned to only one transportation.

Schema Example:

Transportations Table

id	order_id	address
1	1	U.S
2	2	India

Orders Table

id	note
1	some information
2	some comments

Note: In a One-to-One relationship, the foreign key (order_id or transportation_id) can be placed in either table.

2. One to Many

Concept: A single record in Table A can relate to multiple records in Table B, but each record in Table B links back to only one record in Table A.

Example: A user can place multiple orders, but each order belongs to only one user.

Schema Example:

Users Table

id	name	age
1	John	40
2	May	30

Orders Table

id	user_id	note
1	1	some information
2	2	some comments
3	2	no comments

Note: The foreign key (user_id) must reside in the child table (orders). Placing order_id in users would incorrectly restrict each user to a single order.

3. Many to Many

Concept: Multiple records in Table A relate to multiple records in Table B. This requires an intermediate/junction table to map the relationships.

Example: An order can contain multiple products, and a product can be included in multiple orders.

Schema Example:

Orders Table

id	user_id	note
1	1	some information
2	2	some comments
3	2	no comments

Products Table

id	name
1	table
2	chair
3	apple

orders_products (Junction Table)

id	order_id	product_id
1	1	1
2	1	2
3	2	2
4	3	1
5	3	3

Meaningful Transformation (Order Items)

In practice, junction tables can be expanded with additional attributes to hold domain details. Transforming orders_products into order_items allows recording attributes like quantity:

order_items Table

id	order_id	product_id	quantity
1	1	1	1
2	1	2	3
3	2	2	5
4	3	1	3
5	3	3	8

Part 2: SQL JOIN

What You Will Learn

- How to combine tables using LEFT JOIN, RIGHT JOIN, and INNER JOIN.
- How to select specific columns, resolving name collisions.

Sample Data Schema

```
CREATE SCHEMA new_schema DEFAULT CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
```

```
CREATE TABLE `new_schema`.`users` (  
  `id` INT NOT NULL AUTO_INCREMENT COMMENT 'Primary index',  
  `name` VARCHAR(45) NOT NULL DEFAULT 'N/A',  
  `age` INT NULL,  
  PRIMARY KEY (`id`)  
);
```

```
INSERT INTO `new_schema`.`users` (`id`, `name`, `age`) VALUES  
(1, 'John', 40),  
(2, 'May', 30),  
(3, 'Tim', 25);
```

```
CREATE TABLE `new_schema`.`orders` (
  `id` INT NOT NULL AUTO_INCREMENT,
  `user_id` INT,
  `note` VARCHAR(255) NOT NULL,
  PRIMARY KEY (`id`)
);

INSERT INTO `new_schema`.`orders` (`id`, `user_id`, `note`) VALUES
(1, 1, 'some info'),
(2, 2, 'some comments'),
(3, 2, 'no comments'),
(4, NULL, 'weird');
```

1. LEFT JOIN

Keeps all records from the left (main) table and fetches matching records from the right (attached) table. If no match exists, NULL values are returned for the right table's columns.

```
SELECT * FROM `new_schema`.`users`
LEFT JOIN `new_schema`.`orders` ON `users`.`id` = `orders`.`user_id`;
```

Result:

users.id	name	age	orders.id	user_id	note
1	John	40	1	1	some information
2	May	30	2	2	some comments
2	May	30	3	2	no comments
3	Tim	25	NULL	NULL	NULL

2. RIGHT JOIN

Keeps all records from the right (main) table and fetches matching records from the left table.

```
SELECT * FROM `new_schema`.`users`
RIGHT JOIN `new_schema`.`orders` ON `users`.`id` = `orders`.`user_id`;
```

Result:

users.id	name	age	orders.id	user_id	note
1	John	40	1	1	some information
2	May	30	2	2	some comments
2	May	30	3	2	no comments
NULL	NULL	NULL	4	NULL	weird

Equivalence: A RIGHT JOIN can be written as a LEFT JOIN by swapping table order.

3. INNER JOIN

Fetches only records that have matching keys in both tables (intersection).
`SELECT * FROM `new_schema`.`users``

`INNER JOIN `new_schema`.`orders` ON `users`.`id` = `orders`.`user_id`;`

Result:

users.id	name	age	orders.id	user_id	note
1	John	40	1	1	some information
2	May	30	2	2	some comments
2	May	30	3	2	no comments

Selecting Specific Columns (Handling Ambiguity)

When column names overlap between tables (e.g., id), explicitly specify table_name.column_name and use aliases (AS).
`SELECT `orders`.`id` AS order_id, `name``

`FROM `new_schema`.`users``

`INNER JOIN `new_schema`.`orders` ON `users`.`id` = `orders`.`user_id`;`

Part 3: Subqueries**What You Will Learn**

- How to use subqueries with = (equal) and IN (contain) operators.
- How to avoid common subquery errors.

1. Equal Condition (=)

Used when the subquery returns exactly **one** single value/row.
`SELECT * FROM `new_schema`.`orders``

```
WHERE user_id = (  
    SELECT id FROM `new_schema`.`users`  
    WHERE name = 'John'  
);
```

2. Contain Condition (IN)

Used when the subquery returns **multiple** values/rows.
`SELECT * FROM `new_schema`.`orders``

```
WHERE user_id IN (  
    SELECT id FROM `new_schema`.`users`  
    WHERE name LIKE '%j%'  
);
```

Common Subquery Mistakes

- **Singular vs. Plural Mismatch:** Using = when a subquery returns multiple rows causes a runtime error. Use IN instead.
- **Redundant Columns:** Selecting multiple columns in a subquery (e.g., SELECT *) when only one column (e.g., SELECT id) is expected by the comparison operator leads to execution errors.

Movies, OTT & Entertainment – SQL Major Project Dataset

This dataset is **student-friendly**, **industry-inspired**, and suitable for a **major SQL project**. It supports **single-table** → **multi-table learning**, analytics, and reporting.

1. MOVIES TABLE

Table Structure

```
CREATE TABLE movies (  movie_id
INT PRIMARY KEY,  movie_name
VARCHAR(100),  genre
VARCHAR(30),  language
VARCHAR(30),  release_year INT,
duration INT, -- minutes  rating
DECIMAL(3,1) );
```

Insert Data

```
INSERT INTO movies VALUES
(1,'Jai Bhim','Drama','Tamil',2021,164,8.8),
(2,'RRR','Action','Telugu',2022,187,8.6),
(3,'KGF Chapter 2','Action','Kannada',2022,168,8.4),
(4,'Vikram','Action','Tamil',2022,175,8.3),
(5,'Soorai Pottru','Drama','Tamil',2020,153,8.7),
(6,'Drishyam 2','Thriller','Malayalam',2021,152,8.2),
(7,'Dangal','Sports','Hindi',2016,161,8.4),
(8,'Baahubali 2','Fantasy','Telugu',2017,171,8.2),
(9,'Andhadhun','Thriller','Hindi',2018,139,8.2),
(10,'Kantara','Drama','Kannada',2022,148,8.3),
(11,'Leo','Action','Tamil',2023,164,7.8),
(12,'Jawan','Action','Hindi',2023,169,7.9),
(13,'Pushpa','Action','Telugu',2021,179,7.6),
(14,'3 Idiots','Comedy','Hindi',2009,170,8.4),
(15,'Premam','Romance','Malayalam',2015,156,8.3);
```

2. ACTORS TABLE

```
CREATE TABLE actors (  actor_id  
INT PRIMARY KEY,  actor_name  
VARCHAR(50),  gender  
VARCHAR(10) );
```

```
INSERT INTO actors VALUES
```

```
(1,'Surya','Male'),  
(2,'Vijay','Male'),  
(3,'Kamal Haasan','Male'),  
(4,'Yash','Male'),  
(5,'Ram Charan','Male'),  
(6,'Jr NTR','Male'),  
(7,'Aamir Khan','Male'),  
(8,'Prithviraj','Male'),  
(9,'Rajkummar Rao','Male'),  
(10,'Alia Bhatt','Female');
```

3. MOVIE_CAST TABLE (Many-to-Many)

```
CREATE TABLE movie_cast (  
movie_id INT,  actor_id INT,  
role_name VARCHAR(50),  
  PRIMARY KEY (movie_id, actor_id) );
```

```
INSERT INTO movie_cast VALUES
```

```
(1,1,'Lawyer'),  
(2,5,'Freedom Fighter'),  
(2,6,'Revolutionary'),  
(3,4,'Rocky'),  
(4,3,'Agent Vikram'),  
(5,1,'Pilot'),  
(6,8,'Father'),  
(7,7,'Wrestler'),  
(8,5,'Prince'),  
(9,9,'Blind Pianist'),  
(10,4,'Tribal Hero'),  
(11,2,'Gangster'),  
(12,2,'Vigilante'),  
(13,4,'Smuggler'),  
(14,7,'Student'),  
(15,8,'College Student');
```

4. OTT PLATFORMS TABLE

```
CREATE TABLE ott_platforms (  
platform_id INT PRIMARY KEY,  
platform_name VARCHAR(30),  
launch_year INT );
```

```
INSERT INTO ott_platforms VALUES
```

```
(1,'Netflix',2016),  
(2,'Amazon Prime',2016),  
(3,'Disney+ Hotstar',2019),  
(4,'Zee5',2018),  
(5,'Sony LIV',2020);
```

5. MOVIE_STREAMING TABLE

```
CREATE TABLE movie_streaming (  
movie_id INT, platform_id INT,  
streaming_year INT,  
    PRIMARY KEY (movie_id, platform_id)  
);
```

```
INSERT INTO movie_streaming VALUES
```

```
(1,2,2021),  
(2,1,2022),  
(3,2,2022),  
(4,3,2022),  
(5,2,2020),  
(6,3,2021),  
(7,1,2017),  
(8,1,2018),  
(9,2,2019),  
(10,4,2022),  
(11,3,2023),  
(12,1,2023),  
(13,2,2021),  
(14,1,2010),  
(15,2,2016);
```

6. USERS TABLE

```
CREATE TABLE users ( user_id  
INT PRIMARY KEY, user_name
```

```
VARCHAR(50),    city VARCHAR(30)
);
```

```
INSERT INTO users VALUES
```

```
(1,'Arun','Chennai'),
(2,'Divya','Coimbatore'),
(3,'Rahul','Bangalore'),
(4,'Sneha','Hyderabad'),
(5,'Karthik','Mumbai');
```

7. VIEWING HISTORY TABLE

```
CREATE TABLE viewing_history (
view_id INT PRIMARY KEY,    user_id
INT,    movie_id INT,    watch_date
DATE,    watch_hours DECIMAL(3,1) );
```

```
INSERT INTO viewing_history VALUES
```

```
(1,1,1,'2023-01-10',2.5),
(2,1,4,'2023-02-12',3.0),
(3,2,2,'2023-03-15',2.8),
(4,3,3,'2023-04-18',3.1),
(5,4,5,'2023-05-20',2.6),
(6,5,7,'2023-06-25',2.7),
(7,2,10,'2023-07-01',2.4),
(8,3,11,'2023-08-05',2.9);
```

PART A – Single Table Queries (Movies / OTT / Users)

1. Display all movies.

```
SELECT * FROM movies;
```

	movie_id	movie_name	genre	language	release_year	duration	rating
▶	1	Jai Bhim	Drama	Tamil	2021	164	8.8
	2	RRR	Action	Telugu	2022	187	8.6
	3	KGF Chapter 2	Action	Kannada	2022	168	8.4
	4	Vikram	Action	Tamil	2022	175	8.3
	5	Soorai Pottru	Drama	Tamil	2020	153	8.7
	6	Drishyam 2	Thriller	Malayalam	2021	152	8.2
	7	Dangal	Sports	Hindi	2016	161	8.4
	8	Baahubali 2	Fantasy	Telugu	2017	171	8.2
	9	Andhadhun	Thriller	Hindi	2018	139	8.2
	10	Kantara	Drama	Kannada	2022	148	8.3
	11	Leo	Action	Tamil	2023	164	7.8
	12	Jawan	Action	Hindi	2023	169	7.9
	13	Pushpa	Action	Telugu	2021	179	7.6
	14	3 Idiots	Comedy	Hindi	2009	170	8.4
	15	Premam	Romance	Malayalam	2015	156	8.3
★	NULL	NULL	NULL	NULL	NULL	NULL	NULL

2. List movie name, genre, and rating.

```
SELECT movie_name, genre, rating FROM movies;
```

	movie_name	genre	rating
▶	Jai Bhim	Drama	8.8
	RRR	Action	8.6
	KGF Chapter 2	Action	8.4
	Vikram	Action	8.3
	Soorai Pottru	Drama	8.7
	Drishyam 2	Thriller	8.2
	Dangal	Sports	8.4
	Baahubali 2	Fantasy	8.2
	Andhadhun	Thriller	8.2
	Kantara	Drama	8.3
	Leo	Action	7.8
	Jawan	Action	7.9
	Pushpa	Action	7.6
	3 Idiots	Comedy	8.4
	Premam	Romance	8.3

3. Display all action movies.

```
SELECT * FROM movies WHERE genre = 'Action';
```

	movie_id	movie_name	genre	language	release_year	duration	rating
▶	2	RRR	Action	Telugu	2022	187	8.6
	3	KGF Chapter 2	Action	Kannada	2022	168	8.4
	4	Vikram	Action	Tamil	2022	175	8.3
	11	Leo	Action	Tamil	2023	164	7.8
	12	Jawan	Action	Hindi	2023	169	7.9
	13	Pushpa	Action	Telugu	2021	179	7.6
•	NULL	NULL	NULL	NULL	NULL	NULL	NULL

4. List movies released after 2020.

```
SELECT * FROM movies WHERE release_year > 2020;
```

	movie_id	movie_name	genre	language	release_year	duration	rating
▶	1	Jai Bhim	Drama	Tamil	2021	164	8.8
	2	RRR	Action	Telugu	2022	187	8.6
	3	KGF Chapter 2	Action	Kannada	2022	168	8.4
	4	Vikram	Action	Tamil	2022	175	8.3
	6	Drishyam 2	Thriller	Malayalam	2021	152	8.2
	10	Kantara	Drama	Kannada	2022	148	8.3
	11	Leo	Action	Tamil	2023	164	7.8
	12	Jawan	Action	Hindi	2023	169	7.9
	13	Pushpa	Action	Telugu	2021	179	7.6
•	NULL	NULL	NULL	NULL	NULL	NULL	NULL

5. Display movies with rating above 8.5.

```
SELECT * FROM movies WHERE rating > 8.5;
```

	movie_id	movie_name	genre	language	release_year	duration	rating
▶	1	Jai Bhim	Drama	Tamil	2021	164	8.8
	2	RRR	Action	Telugu	2022	187	8.6
	5	Soorarai Pottru	Drama	Tamil	2020	153	8.7

6. Show distinct movie languages.

```
SELECT DISTINCT language FROM movies;
```

	language
▶	Tamil
	Telugu
	Kannada
	Malayalam
	Hindi

7. Display top 5 highest-rated movies.

```
SELECT * FROM movies ORDER BY rating DESC LIMIT 5;
```

	movie_id	movie_name	genre	language	release_year	duration	rating
▶	1	Jai Bhim	Drama	Tamil	2021	164	8.8
	5	Soorarai Pottru	Drama	Tamil	2020	153	8.7
	2	RRR	Action	Telugu	2022	187	8.6
	3	KGF Chapter 2	Action	Kannada	2022	168	8.4
	7	Dangal	Sports	Hindi	2016	161	8.4

8. Count total number of movies.

```
SELECT COUNT(*) FROM movies;
```

	total_movies
▶	15

9. Find average movie duration.

```
SELECT AVG(duration) FROM movies;
```

	average_duration
▶	163.7333

10. Display movies ordered by release year.

```
SELECT * FROM movies ORDER BY release_year;
```

	movie_id	movie_name	genre	language	release_year	duration	rating
▶	14	3 Idiots	Comedy	Hindi	2009	170	8.4
	15	Premam	Romance	Malayalam	2015	156	8.3
	7	Dangal	Sports	Hindi	2016	161	8.4
	8	Baahubali 2	Fantasy	Telugu	2017	171	8.2
	9	Andhadhun	Thriller	Hindi	2018	139	8.2
	5	Soorai Pottru	Drama	Tamil	2020	153	8.7
	1	Jai Bhim	Drama	Tamil	2021	164	8.8
	6	Drishyam 2	Thriller	Malayalam	2021	152	8.2
	13	Pushpa	Action	Telugu	2021	179	7.6
	2	RRR	Action	Telugu	2022	187	8.6
	3	KGF Chapter 2	Action	Kannada	2022	168	8.4
	4	Vikram	Action	Tamil	2022	175	8.3
	10	Kantara	Drama	Kannada	2022	148	8.3
	11	Leo	Action	Tamil	2023	164	7.8
	12	Jawan	Action	Hindi	2023	169	7.9

PART B – GROUP BY & Aggregates

11. Genre-wise movie count.

```
SELECT genre, COUNT(*) FROM movies GROUP BY genre;
```

	genre	movie_count
▶	Drama	3
	Action	6
	Thriller	2
	Sports	1
	Fantasy	1
	Comedy	1
	Romance	1

12. Language-wise average rating.

```
SELECT language, AVG(rating) FROM movies GROUP BY language;
```

	language	average_rating
▶	Tamil	8.40000
	Telugu	8.13333
	Kannada	8.35000
	Malayalam	8.25000
	Hindi	8.22500

13. Find maximum rating per genre.

```
SELECT genre, MAX(rating) FROM movies GROUP BY genre;
```

	genre	maximum_rating
▶	Drama	8.8
	Action	8.6
	Thriller	8.2
	Sports	8.4
	Fantasy	8.2
	Comedy	8.4
	Romance	8.3

14.Count movies released each year.

```
SELECT release_year, COUNT(*) FROM movies GROUP BY release_year;
```

	release_year	movis_count_per_year
▶	2021	3
	2022	4
	2020	1
	2016	1
	2017	1
	2018	1
	2023	2
	2009	1
	2015	1

15.Display genres having more than 2 movies.

```
SELECT genre, COUNT(*) FROM movies GROUP BY genre HAVING COUNT(*) > 2;
```

	genre	movies_count
▶	Drama	3
	Action	6

PART C – JOIN Queries (Movies, OTT, Streaming)

16. List movies with their OTT platforms.

```
SELECT m.movie_name, o.platform_name  
FROM movies m  
JOIN movie_streaming ms ON m.movie_id = ms.movie_id  
JOIN ott_platforms o ON ms.platform_id = o.platform_id;
```

	movie_name	platform_name
▶	Jai Bhim	Amazon Prime
	RRR	Netflix
	KGF Chapter 2	Amazon Prime
	Vikram	Disney + Hotstar
	Soorarai Pottru	Amazon Prime
	Drishyam 2	Disney + Hotstar
	Dangal	Netflix
	Baahubali 2	Netflix
	Andhadhun	Amazon Prime
	Kantara	Zee5
	Leo	Disney + Hotstar
	Jawan	Netflix
	Pushpa	Amazon Prime
	3 Idiots	Netflix
	Premam	Amazon Prime

17. Display movies available on Netflix.

```
SELECT m.movie_name  
FROM movies m  
JOIN movie_streaming ms ON m.movie_id = ms.movie_id  
JOIN ott_platforms o ON ms.platform_id = o.platform_id  
WHERE o.platform_name = 'Netflix';
```

	movie_name
▶	RRR
	Dangal
	Baahubali 2
	Jawan
	3 Idiots

18.Count movies per OTT platform.

```
SELECT o.platform_name, COUNT(*)  
  
FROM ott_platforms o  
  
JOIN movie_streaming ms ON o.platform_id = ms.platform_id  
  
GROUP BY o.platform_name;
```

	platform_name	COUNT(*)
▶	Amazon Prime	6
	Netflix	5
	Disney+ Hotstar	3
	Zee5	1

19. Find OTT platform with maximum movies.

```
SELECT o.platform_name, COUNT(*) AS total  
  
FROM ott_platforms o  
  
JOIN movie_streaming ms ON o.platform_id = ms.platform_id  
  
GROUP BY o.platform_name  
  
ORDER BY total DESC  
  
LIMIT 1;
```

	platform_name	total
▶	Amazon Prime	6

20.Display movies streamed after 2021.

```
SELECT m.movie_name  
  
FROM movies m  
  
JOIN movie_streaming ms ON m.movie_id = ms.movie_id  
  
WHERE ms.streaming_year > 2021;
```

	movie_name
▶	RRR
	KGF Chapter 2
	Vikram
	Kantara
	Leo
	Jawan

PART D – Actor & Cast Analysis

21. List actors and their movies.

```
SELECT a.actor_name, m.movie_name
FROM actors a
JOIN movie_cast mc ON a.actor_id = mc.actor_id
JOIN movies m ON mc.movie_id = m.movie_id;
```

	actor_name	movie_name
▶	Surya	Jai Bhim
	Ram Charan	RRR
	Jr NTR	RRR
	Yash	KGF Chapter 2
	Kamal Haasan	Vikram
	Surya	Soorai Pottru
	Prithviraj	Drishyam 2
	Aamir Khan	Dangal
	Ram Charan	Baahubali 2
	Rajkummar Rao	Andhadhun
	Yash	Kantara
	Vijay	Leo
	Vijay	Jawan
	Yash	Pushpa

22.Count number of movies per actor.

```
SELECT a.actor_name, COUNT(*)
FROM actors a
JOIN movie_cast mc ON a.actor_id = mc.actor_id
GROUP BY a.actor_name;
```

	actor_name	COUNT(*)
▶	Surya	2
	Ram Charan	2
	Jr NTR	1
	Yash	3
	Kamal Haasan	1
	Prithviraj	2
	Aamir Khan	2
	Rajkummar Rao	1
	Vijay	2

23. Actor who acted in maximum movies.

```
SELECT a.actor_name, COUNT(*) AS total
FROM actors a
JOIN movie_cast mc ON a.actor_id = mc.actor_id
GROUP BY a.actor_name
ORDER BY total DESC
LIMIT 1;
```

	actor_name	total
►	Yash	3

24. List movies featuring actor 'Yash'.

```
SELECT m.movie_name
FROM movies m
JOIN movie_cast mc ON m.movie_id = mc.movie_id
JOIN actors a ON mc.actor_id = a.actor_id
WHERE a.actor_name = 'Yash';
```

	movie_name
►	KGF Chapter 2
	Kantara
	Pushpa

25. Find actors involved in action movies.

```
SELECT DISTINCT a.actor_name
FROM actors a
JOIN movie_cast mc ON a.actor_id = mc.actor_id
JOIN movies m ON mc.movie_id = m.movie_id
WHERE m.genre = 'Action';
```

	actor_name
►	Ram Charan
	Jr NTR
	Yash
	Kamal Haasan
	Vijay

PART E – User Viewing Analysis

26. List users and movies they watched.

```
SELECT u.user_name, m.movie_name  
  
FROM users u  
  
JOIN viewing_history v ON u.user_id = v.user_id  
  
JOIN movies m ON v.movie_id = m.movie_id;
```

	user_name	movie_name
▶	Arun	Jai Bhim
	Arun	Vikram
	Divya	RRR
	Rahul	KGF Chapter 2
	Sneha	Soorarai Pottru
	Karthik	Dangal
	Divya	Kantara
	Rahul	Leo

27.Count movies watched by each user.

```
SELECT u.user_name, COUNT(*)  
  
FROM users u  
  
JOIN viewing_history v ON u.user_id = v.user_id  
  
GROUP BY u.user_name;
```

	user_name	COUNT(*)
▶	Arun	2
	Divya	2
	Rahul	2
	Sneha	1
	Karthik	1

28.User who watched maximum movies.

```
SELECT u.user_name, COUNT(*) AS total  
  
FROM users u  
  
JOIN viewing_history v ON u.user_id = v.user_id  
  
GROUP BY u.user_name  
  
ORDER BY total DESC  
  
LIMIT 1;
```

	user_name	total
▶	Arun	2

29. Most watched movie.

```
SELECT m.movie_name, COUNT(*) AS views
FROM movies m
JOIN viewing_history v ON m.movie_id = v.movie_id
GROUP BY m.movie_name
ORDER BY views DESC
LIMIT 1;
```

	movie_name	views
▶	Jai Bhim	1

30. Total watch hours per movie.

```
SELECT m.movie_name, SUM(v.watch_hours)
FROM movies m
JOIN viewing_history v ON m.movie_id = v.movie_id
GROUP BY m.movie_name;
```

	movie_name	SUM(v.watch_hours)
▶	Jai Bhim	2.5
	Vikram	3.0
	RRR	2.8
	KGF Chapter 2	3.1
	Soorarai Pottru	2.6
	Dangal	2.7
	Kantara	2.4
	Leo	2.9

PART F – Subqueries & Analytics

31. Movies with rating above average.

```
SELECT * FROM movies  
WHERE rating > (SELECT AVG(rating) FROM movies);
```

	movie_id	movie_name	genre	language	release_year	duration	rating
▶	1	Jai Bhim	Drama	Tamil	2021	164	8.8
	2	RRR	Action	Telugu	2022	187	8.6
	3	KGF Chapter 2	Action	Kannada	2022	168	8.4
	4	Vikram	Action	Tamil	2022	175	8.3
	5	Soorarai Pottru	Drama	Tamil	2020	153	8.7
	7	Dangal	Sports	Hindi	2016	161	8.4
	10	Kantara	Drama	Kannada	2022	148	8.3
	14	3 Idiots	Comedy	Hindi	2009	170	8.4
	15	Premam	Romance	Malayalam	2015	156	8.3

32. Movies longer than average duration.

```
SELECT * FROM movies  
WHERE duration > (SELECT AVG(duration) FROM movies);
```

	movie_id	movie_name	genre	language	release_year	duration	rating
▶	1	Jai Bhim	Drama	Tamil	2021	164	8.8
	2	RRR	Action	Telugu	2022	187	8.6
	3	KGF Chapter 2	Action	Kannada	2022	168	8.4
	4	Vikram	Action	Tamil	2022	175	8.3
	8	Baahubali 2	Fantasy	Telugu	2017	171	8.2
	11	Leo	Action	Tamil	2023	164	7.8
	12	Jawan	Action	Hindi	2023	169	7.9
	13	Pushpa	Action	Telugu	2021	179	7.6
	14	3 Idiots	Comedy	Hindi	2009	170	8.4

33. OTT platforms hosting action movies.

```
SELECT DISTINCT o.platform_name  
FROM ott_platforms o  
JOIN movie_streaming ms ON o.platform_id = ms.platform_id  
JOIN movies m ON ms.movie_id = m.movie_id  
WHERE m.genre = 'Action';
```

	platform_name
▶	Netflix
	Amazon Prime
	Disney + Hotstar

34.Users who watched movies rated above 8.5.

```
SELECT DISTINCT u.user_name
FROM users u
JOIN viewing_history v ON u.user_id = v.user_id
JOIN movies m ON v.movie_id = m.movie_id
WHERE m.rating > 8.5;
```

	user_name
▶	Arun
	Divya
	Sneha

35.Movies not yet watched by any user.

```
SELECT movie_name FROM movies
WHERE movie_id NOT IN (SELECT movie_id FROM viewing_history);
```

	movie_name
▶	Drishyam 2
	Baahubali 2
	Andhadhun
	Jawan
	Pushpa
	3 Idiots
	Premam

PART G – Date & Reporting Queries

36. Movies watched in 2023.

```
SELECT DISTINCT m.movie_name
FROM movies m
JOIN viewing_history v ON m.movie_id = v.movie_id
WHERE YEAR(v.watch_date) = 2023;
```

	movie_name
▶	Jai Bhim
	Vikram
	RRR
	KGF Chapter 2
	Soorai Pottru
	Dangal
	Kantara
	Leo

37. Users who watched movies in multiple months.

```
SELECT u.user_name
FROM users u
JOIN viewing_history v ON u.user_id = v.user_id
GROUP BY u.user_name
HAVING COUNT(DISTINCT MONTH(v.watch_date)) > 1;
```

	user_name
▶	Arun
	Divya
	Rahul

38. Movies streamed before release year.

```
SELECT m.movie_name
FROM movies m
JOIN movie_streaming ms ON m.movie_id = ms.movie_id
WHERE ms.streaming_year < m.release_year;
```

	movie_name
--	------------

39. Latest released movie.

```
SELECT * FROM movies ORDER BY release_year DESC LIMIT 1;
```

	movie_id	movie_name	genre	language	release_year	duration	rating
▶	11	Leo	Action	Tamil	2023	164	7.8

40. Oldest movie in dataset.

```
SELECT * FROM movies ORDER BY release_year LIMIT 1;
```

	movie_id	movie_name	genre	language	release_year	duration	rating
▶	14	3 Idiots	Comedy	Hindi	2009	170	8.4